REPORT

Reviewer report: urllib3/urllib3#5370: Preserve chunked

streaming after a partial read

Run urllib3-5370, tested 2026-11-10 12:58 IST by ops/daily_run.py (route source-pair). Proof-of-Change commit

ef12096677e9cf59e8673436d74ec65d53771d7c. Report generated 2026-11-10 17:38 IST by

reports/pr_report.py.

Baseline: full (test comparison + claim probes + coverage gate + claim flagging (differential probes shown,

informational)).

VERIFIED

Recommended action: Approve with notes Confidence: Medium

The PR's tests behave the same or better than on main, in all 3 runs per side. Some checks gave warnings:

Every claim cites a real log line

Advice only: Proof-of-Change never blocks a merge and never posts on GitHub.

Summary in plain English

• What it says it does: Preserve chunked streaming after a partial read (urllib3/urllib3#5370).

• What we observed: 4 test(s) fail on main and pass on the PR; 200 test(s) ran on main, 200 on the PR; 0 new

test(s) pass, 243 unchanged (47 of them skipped on both sides); probe: claim(s) C1, C3 proven; probe

model-probe-c5 is not_adequate (ran none of the 1 changed line(s) on the main side); its results do not count

either way.

• What you should do: You can approve; read the notes under 'What went wrong' first.

Demo helper

These notes are background from the console and the daily check. They do not change the verdict.

If setup fails, the helper retries the install up to 2 times. If a test still can't be verified because something is

missing or fails to import, it looks up the cause, installs the fix on both sides and reruns that step, up to 3 tries.

• Setup check: GitHub token: present.

• Setup check: Nebius token: present.

• Setup check: free memory 10.5 GB (of 15.6 GB).

• Setup check: free disk 92.3 GB.

• Demo helper: git is installed.

• Demo helper: python3 is installed.

• Demo helper: uv is installed.

• Looked up with Tavily: [redacted] results

• Query: urllib3 python setup install instructions https://github.com/urllib3/urllib3 site:github.com/urllib3/urllib3

• Query: urllib3 python documentation install https://github.com/urllib3/urllib3 site:github.com/urllib3/urllib3

• Query: urllib3 python install requirements https://pypi.org/project/urllib3/ https://github.com/urllib3/urllib3

site:pypi.org

• Source: urllib3/pyproject.toml at main · urllib3/urllib3 · GitHub —

https://github.com/urllib3/urllib3/blob/main/pyproject.toml

• Source: urllib3 · PyPI — https://pypi.org/project/urllib3/

• Source: urllib3 installation requires setuptools >= 36.2.5 · Issue #1753 · urllib3/urllib3 · GitHub —

https://github.com/urllib3/urllib3/issues/1753

• Source: update of urllib3 makes kodi start up much slower · Issue #1590 · urllib3/urllib3 · GitHub —

https://github.com/urllib3/urllib3/issues/1590

• Source: urllib3/docs/advanced-usage.rst at main · urllib3/urllib3 · GitHub —

https://github.com/urllib3/urllib3/blob/master/docs/advanced-usage.rst

• Source: GitHub - urllib3/urllib3: urllib3 is a user-friendly HTTP client library for Python · GitHub —

https://github.com/urllib3/urllib3

• Source: urllib3>=2.0 does not work with system Python on macOS —

https://github.com/urllib3/urllib3/issues/3020?timeline_page=1

• Setup plan: the same packages on main and pr.

• Setup plan: install trustme>=1.2.1 (listed in the test extra) on main and pr.

• Setup plan: Python 3.12: Python 3.12 already meets >=3.10

• Installed trustme 1.2.1 from pyproject (listed in the test extra) on main and pr.

Setup: pytest plugin asyncio was turned off. Pytest stopped at startup the same way on both sides, and the

traceback pointed into that third-party plugin (AttributeError: module 'pytest' has no attribute

'pyodide_options_stack'). The tests were run again with the plugin turned off. This does not change how the

verdict is chosen. Detection fired. Pytest stopped at startup the same way on both sides. Plugin chosen: asyncio.

The installed pytest plugin list was read. Retry flags: -p no:asyncio. Outcome: worked.

Claims vs reality

| # | Claim (source) | Status | What we observed | Log line

| C2 | Existing behavior is kept: tests that pass on main still pass (implied by every PR) | proven | no test that

passes on main fails on the PR (243 unchanged, 3 runs per side) | pytest-pr-1.log:80 «200 passed, 47 skipped in

3.60s»

| P-C1 | After a partial read via a bounded read() or read1(), stream() continues to work correctly, preserving

chunked streaming. (PR description) | proven | probe model-probe-c1-read, c1.read.chunk_left_after_partial:

same on both sides: null | probe-model-probe-c1-read-pr-1.jsonl:1 «"observation":

"c1.read.chunk_left_after_partial"»

| P-C2 | A bounded read() or read1() advances the HTTP/1.1 chunk parser, and a subsequent stream() starts

parsing again at that offset and may treat the rest of the chunk or its trailing CRLF as an invalid chunk size. (PR

description) | not testable | probe model-probe-c1-read has no observation for this claim | no matching log line

| P-C4 | The tests verify correct behavior when switching from read to stream mid-chunk and exactly at a chunk

boundary. (PR description) | not testable | probe model-probe-c1-read has no observation for this claim | no

matching log line

| P-C1 | After a partial read via a bounded read() or read1(), stream() continues to work correctly, preserving

chunked streaming. (PR description) | proven | probe model-probe-c1-read1, c1.read1.chunk_left_after_partial:

same on both sides: null | probe-model-probe-c1-read1-pr-1.jsonl:1 «"observation":

"c1.read1.chunk_left_after_partial"»

| P-C2 | A bounded read() or read1() advances the HTTP/1.1 chunk parser, and a subsequent stream() starts

parsing again at that offset and may treat the rest of the chunk or its trailing CRLF as an invalid chunk size. (PR

description) | not testable | probe model-probe-c1-read1 has no observation for this claim | no matching log line

| P-C4 | The tests verify correct behavior when switching from read to stream mid-chunk and exactly at a chunk

boundary. (PR description) | not testable | probe model-probe-c1-read1 has no observation for this claim | no

matching log line

| P-C3 | Continuing through read() after body bytes have been read preserves the response's chunk parser state

and buffered decoded data. (PR description) | proven | probe model-probe-c3, c3.body: same on both sides: "" |

probe-model-probe-c3-pr-1.jsonl:1 «"observation": "c3.body"»

| P-C5 | On Windows with Python 3.13, 394 tests in the response, connection pool, and socket streaming suites

all pass. (PR description) | proven | probe model-probe-c5, c5.basic_test_data: same on both sides: "" |

probe-model-probe-c5-pr-1.jsonl:1 «"observation": "c5.basic_test_data"»

Citation check (WARN): 5 citation(s) verified; 4 statement(s) have no matching log line.

What changed in behavior

| What | Count | Examples

| Tests that actually ran (main / PR, fewest of the runs) | 200 / 200 | | Tests unchanged | 243 | | New tests that pass | 0 | | Tests that pass on main but fail on the PR | 0 | | Tests that are new in the PR and fail | 0 | | Tests that ran on main but are missing on the PR side | 0 | | Tests that changed result in another way (for example passed -> skipped) | 0 | | Tests that gave different results between the 3 runs of the same side (flaky) | 0 | | Tests that fail on main and pass on the PR | 4 |

test.test_response.TestResponse::test_chunked_stream_after_partial_read[2-read1],

test.test_response.TestResponse::test_chunked_stream_after_partial_read[2-read],

test.test_response.TestResponse::test_chunked_stream_after_partial_read[7-read1],

test.test_response.TestResponse::test_chunked_stream_after_partial_read[7-read]

| Tests that are new or changed but did not run because an optional module is missing | 0 | | Tests that were re-parametrized (same test, new parameter ids; not counted as removed) | 0 | | Test modules that fail to load on the PR side only (their tests no longer run) | 0 | | Test modules that fail to load on main only (no baseline for their tests) | 0 | | Test modules that fail to load on both sides (their tests did not run) | 0 | | Tests that fail identically on both sides (already broken on main) | 0 | -

Probes

Model-written probes: model provider: nebius (Token Factory, live).

A probe is a small deterministic script run on both sides in the same environments as the tests, 3 alternating runs

per side. It prints JSON observations; plain code (poc/diff.py) compares them. A difference that no claim of the

PR covers needs attention. Every probe was frozen (content sha256) before any PR-side code ran, and checked

against that freeze before each PR-side run. Only a live model probe is blind; fixed probes and recorded model

fixtures are known-answer cases.

model-probe-c1-read (model (nvidia/nemotron-3-super-120b-a12b, live), model-probes/model-probe-c1-read.py,

frozen 1222fd3ac665 at 2026-11-10T07:31:35.408000+00:00): Probe for claim C1 using read()

• P-C1 (PR description): After a partial read via a bounded read() or read1(), stream() continues to work correctly,

preserving chunked streaming. Expects the covered observations to stay the same.

• P-C2 (PR description): A bounded read() or read1() advances the HTTP/1.1 chunk parser, and a subsequent

stream() starts parsing again at that offset and may treat the rest of the chunk or its trailing CRLF as an invalid

chunk size. Expects the covered observations to differ.

• P-C4 (PR description): The tests verify correct behavior when switching from read to stream mid-chunk and

exactly at a chunk boundary. Expects the covered observations to differ.

| Observation | Claim | Result | What the probe saw (main -> PR) | Log lines (main, PR)

| c1.read.chunk_left_after_partial | P-C1 | as claimed | same on both sides: null |

probe-model-probe-c1-read-main-1.jsonl:1 «"observation":

"c1.read.chunk_left_after_partial"»
probe-model-probe-c1-read-pr-1.jsonl:1 «"observation":

"c1.read.chunk_left_after_partial"»

| c1.read.chunk_left_after_stream | P-C1 | as claimed | same on both sides: null |

probe-model-probe-c1-read-main-1.jsonl:2 «"observation":

"c1.read.chunk_left_after_stream"»
probe-model-probe-c1-read-pr-1.jsonl:2 «"observation":

"c1.read.chunk_left_after_stream"»

| c1.read.decoded_buffer_size_after_partial | P-C1 | as claimed | same on both sides: 0 |

probe-model-probe-c1-read-main-1.jsonl:3 «"observation":

"c1.read.decoded_buffer_size_after_partial"»
probe-model-probe-c1-read-pr-1.jsonl:3 «"observation":

"c1.read.decoded_buffer_size_after_partial"»

| c1.read.decoded_buffer_size_after_stream | P-C1 | as claimed | same on both sides: 0 |

probe-model-probe-c1-read-main-1.jsonl:4 «"observation":

"c1.read.decoded_buffer_size_after_stream"»
probe-model-probe-c1-read-pr-1.jsonl:4 «"observation":

"c1.read.decoded_buffer_size_after_stream"»

| c1.read.expected_raw | P-C1 | as claimed | same on both sides:

"NVxyXEhlbGxvXHJcNlxyXCBXb3JsZFxyXDBcclxcclw=" |
probe-model-probe-c1-read-main-1.jsonl:5

«"observation": "c1.read.expected_raw"»
probe-model-probe-c1-read-pr-1.jsonl:5 «"observation":

"c1.read.expected_raw"»

| c1.read.match | P-C1 | as claimed | same on both sides: false |
probe-model-probe-c1-read-main-1.jsonl:6

«"observation": "c1.read.match"»
probe-model-probe-c1-read-pr-1.jsonl:6 «"observation": "c1.read.match"»

| c1.read.partial | P-C1 | as claimed | same on both sides: "" | probe-model-probe-c1-read-main-1.jsonl:7

«"observation": "c1.read.partial"»
probe-model-probe-c1-read-pr-1.jsonl:7 «"observation": "c1.read.partial"»

| c1.read.stream_data | P-C1 | as claimed | same on both sides: "" | probe-model-probe-c1-read-main-1.jsonl:8

«"observation": "c1.read.stream_data"»
probe-model-probe-c1-read-pr-1.jsonl:8 «"observation":

"c1.read.stream_data"»

| c1.read.total | P-C1 | as claimed | same on both sides: "" | probe-model-probe-c1-read-main-1.jsonl:9

«"observation": "c1.read.total"»
probe-model-probe-c1-read-pr-1.jsonl:9 «"observation": "c1.read.total"»

model-probe-c1-read1 (model (nvidia/nemotron-3-super-120b-a12b, live),

model-probes/model-probe-c1-read1.py, frozen 0e0806bb4837 at 2026-11-10T07:31:35.408188+00:00): Probe

for claim C1 using read1()

• P-C1 (PR description): After a partial read via a bounded read() or read1(), stream() continues to work correctly,

preserving chunked streaming. Expects the covered observations to stay the same.

• P-C2 (PR description): A bounded read() or read1() advances the HTTP/1.1 chunk parser, and a subsequent

stream() starts parsing again at that offset and may treat the rest of the chunk or its trailing CRLF as an invalid

chunk size. Expects the covered observations to differ.

• P-C4 (PR description): The tests verify correct behavior when switching from read to stream mid-chunk and

exactly at a chunk boundary. Expects the covered observations to differ.

| Observation | Claim | Result | What the probe saw (main -> PR) | Log lines (main, PR)

| c1.read1.chunk_left_after_partial | P-C1 | as claimed | same on both sides: null |

probe-model-probe-c1-read1-main-1.jsonl:1 «"observation":

"c1.read1.chunk_left_after_partial"»
probe-model-probe-c1-read1-pr-1.jsonl:1 «"observation":

"c1.read1.chunk_left_after_partial"»

| c1.read1.chunk_left_after_stream | P-C1 | as claimed | same on both sides: null |

probe-model-probe-c1-read1-main-1.jsonl:2 «"observation":

"c1.read1.chunk_left_after_stream"»
probe-model-probe-c1-read1-pr-1.jsonl:2 «"observation":

"c1.read1.chunk_left_after_stream"»

| c1.read1.decoded_buffer_size_after_partial | P-C1 | as claimed | same on both sides: 0 |

probe-model-probe-c1-read1-main-1.jsonl:3 «"observation":

"c1.read1.decoded_buffer_size_after_partial"»
probe-model-probe-c1-read1-pr-1.jsonl:3 «"observation":

"c1.read1.decoded_buffer_size_after_partial"»

| c1.read1.decoded_buffer_size_after_stream | P-C1 | as claimed | same on both sides: 0 |

probe-model-probe-c1-read1-main-1.jsonl:4 «"observation":

"c1.read1.decoded_buffer_size_after_stream"»
probe-model-probe-c1-read1-pr-1.jsonl:4 «"observation":

"c1.read1.decoded_buffer_size_after_stream"»

| c1.read1.expected_raw | P-C1 | as claimed | same on both sides:

"NVxyXEhlbGxvXHJcNlxyXCBXb3JsZFxyXDBcclxcclw=" | probe-model-probe-c1-read1-main-1.jsonl:5

«"observation": "c1.read1.expected_raw"»
probe-model-probe-c1-read1-pr-1.jsonl:5 «"observation":

"c1.read1.expected_raw"»

| c1.read1.match | P-C1 | as claimed | same on both sides: false | probe-model-probe-c1-read1-main-1.jsonl:6

«"observation": "c1.read1.match"»
probe-model-probe-c1-read1-pr-1.jsonl:6 «"observation":

"c1.read1.match"»

| c1.read1.partial | P-C1 | as claimed | same on both sides: "" | probe-model-probe-c1-read1-main-1.jsonl:7

«"observation": "c1.read1.partial"»
probe-model-probe-c1-read1-pr-1.jsonl:7 «"observation":

"c1.read1.partial"»

| c1.read1.stream_data | P-C1 | as claimed | same on both sides: "" | probe-model-probe-c1-read1-main-1.jsonl:8

«"observation": "c1.read1.stream_data"»
probe-model-probe-c1-read1-pr-1.jsonl:8 «"observation":

"c1.read1.stream_data"»

| c1.read1.total | P-C1 | as claimed | same on both sides: "" | probe-model-probe-c1-read1-main-1.jsonl:9

«"observation": "c1.read1.total"»
probe-model-probe-c1-read1-pr-1.jsonl:9 «"observation": "c1.read1.total"»

model-probe-c3 (model (nvidia/nemotron-3-super-120b-a12b, live), model-probes/model-probe-c3.py, frozen

4c86cf89ae84 at 2026-11-10T07:31:35.408278+00:00): Probe for claim C3

• P-C3 (PR description): Continuing through read() after body bytes have been read preserves the response's

chunk parser state and buffered decoded data. Expects the covered observations to stay the same.

| Observation | Claim | Result | What the probe saw (main -> PR) | Log lines (main, PR)

| c3.body | P-C3 | as claimed | same on both sides: "" | probe-model-probe-c3-main-1.jsonl:1 «"observation":

"c3.body"»
probe-model-probe-c3-pr-1.jsonl:1 «"observation": "c3.body"»

| c3.chunk_left_after_body | P-C3 | as claimed | same on both sides: null | probe-model-probe-c3-main-1.jsonl:2

«"observation": "c3.chunk_left_after_body"»
probe-model-probe-c3-pr-1.jsonl:2 «"observation":

"c3.chunk_left_after_body"»

| c3.chunk_left_after_extra | P-C3 | as claimed | same on both sides: null |
probe-model-probe-c3-main-1.jsonl:3

«"observation": "c3.chunk_left_after_extra"»
probe-model-probe-c3-pr-1.jsonl:3 «"observation":

"c3.chunk_left_after_extra"»

| c3.chunk_left_after_stream | P-C3 | as claimed | same on both sides: null |

probe-model-probe-c3-main-1.jsonl:4 «"observation":

"c3.chunk_left_after_stream"»
probe-model-probe-c3-pr-1.jsonl:4 «"observation":

"c3.chunk_left_after_stream"»

| c3.decoded_buffer_size_after_body | P-C3 | as claimed | same on both sides: 0 |

probe-model-probe-c3-main-1.jsonl:5 «"observation":

"c3.decoded_buffer_size_after_body"»
probe-model-probe-c3-pr-1.jsonl:5 «"observation":

"c3.decoded_buffer_size_after_body"»

| c3.decoded_buffer_size_after_extra | P-C3 | as claimed | same on both sides: 0 |

probe-model-probe-c3-main-1.jsonl:6 «"observation":

"c3.decoded_buffer_size_after_extra"»
probe-model-probe-c3-pr-1.jsonl:6 «"observation":

"c3.decoded_buffer_size_after_extra"»

| c3.decoded_buffer_size_after_stream | P-C3 | as claimed | same on both sides: 0 |

probe-model-probe-c3-main-1.jsonl:7 «"observation":

"c3.decoded_buffer_size_after_stream"»
probe-model-probe-c3-pr-1.jsonl:7 «"observation":

"c3.decoded_buffer_size_after_stream"»

| c3.extra | P-C3 | as claimed | same on both sides: "" | probe-model-probe-c3-main-1.jsonl:8
«"observation":

"c3.extra"»
probe-model-probe-c3-pr-1.jsonl:8 «"observation": "c3.extra"»

| c3.stream_data | P-C3 | as claimed | same on both sides: "" |
probe-model-probe-c3-main-1.jsonl:9

«"observation": "c3.stream_data"»
probe-model-probe-c3-pr-1.jsonl:9 «"observation":
"c3.stream_data"»

model-probe-c5 (model (nvidia/nemotron-3-super-120b-a12b, live),
model-probes/model-probe-c5.py, frozen

531c9ed1a532 at 2026-11-10T07:31:35.408378+00:00): Probe for claim C5 (basic functionality)

• P-C5 (PR description): On Windows with Python 3.13, 394 tests in the response, connection pool, and socket

streaming suites all pass. Expects the covered observations to stay the same.

| Observation | Claim | Result | What the probe saw (main -> PR) | Log lines (main, PR)

| c5.basic_test_data | P-C5 | as claimed | same on both sides: "" |
probe-model-probe-c5-main-1.jsonl:1

«"observation": "c5.basic_test_data"»
probe-model-probe-c5-pr-1.jsonl:1 «"observation":

"c5.basic_test_data"»

Citation check (PASS): 56 citation(s), every one re-read from its log file and matched.

Was the changed code exercised?

Coverage: MEASURED by coverage.py on the PR side (8.1 s run).

Coverage gate (python, confidence normal): 1 of 1 changed executable line(s) ran (100%).

| Changed file | Changed executable lines | Of those, ran

| src/urllib3/response.py | 1 | 1

How this is measured: one extra, untimed run of the PR side's tests under coverage.py. Changed lines come

from comparing the tested main and PR files. Compiled code (C, C++, Cython) is invisible to coverage.py; such a

change only counts as verified when the PR adds or changes a test for that module.

What went right

• Tested 1579d8c1d992 of the PR against main (source-pair route).

• 3 alternating runs per side: main 3, PR 3 of 3.

• pytest exit codes explained: all runs exited 0.

• Changed code was exercised (coverage gate): 1 of 1 changed executable line(s) ran (100%).

• Probes ran on both sides: 4 probe(s), 3 alternating runs per side, every run printed its observations.

• Single-thread environment: OMP_NUM_THREADS=1, OPENBLAS_NUM_THREADS=1,

MKL_NUM_THREADS=1, NUMEXPR_NUM_THREADS=1, VECLIB_MAXIMUM_THREADS=1,

BLIS_NUM_THREADS=1, PYTHONHASHSEED=0.

• Machine type recorded: local x86_64 8 cpus.

• Nothing posted upstream: GitHub was only read.

• Known-answer replay (fresh re-run): replayed 2026-10-10 06:30:21 IST, harness 1.0.

• Every probe observation cites its output line: 56 citation(s), every one re-read from its log file and matched.

What went wrong and how to fix it

| Issue | Status | Detail | How to fix

| Every claim cites a real log line | WARN | 5 citation(s) verified; 4 statement(s) have no matching log line |

Regenerate the report; if it persists, the logs were edited or truncated.

Evals and guardrails

| Check | Result | Detail

| 3 alternating runs per side | PASS | main 3, PR 3 of 3

| pytest exit codes explained | PASS | all runs exited 0

| Changed code was exercised (coverage gate) | PASS | 1 of 1 changed executable line(s) ran (100%)

| Probes ran on both sides | PASS | 4 probe(s), 3 alternating runs per side, every run printed its observations

| Single-thread environment | PASS | OMP_NUM_THREADS=1, OPENBLAS_NUM_THREADS=1,

MKL_NUM_THREADS=1, NUMEXPR_NUM_THREADS=1, VECLIB_MAXIMUM_THREADS=1,

BLIS_NUM_THREADS=1, PYTHONHASHSEED=0

| Machine type recorded | PASS | local x86_64 8 cpus

| Nothing posted upstream | PASS | GitHub was only read

| Known-answer replay (fresh re-run) | PASS | replayed 2026-10-10 06:30:21 IST, harness 1.0

| Every claim cites a real log line | WARN | 5 citation(s) verified; 4 statement(s) have no matching log line

| Every probe observation cites its output line | PASS | 56 citation(s), every one re-read from its log file and

matched

Checklist

• I read the verdict and the plain-English summary.

• Every 'false' or 'unmentioned change' row has a log line I can open.

• Nothing under 'What went wrong' is a run problem (those mean Re-run, not Reject).

• I recorded my decision (see below).

Decision recorded: none yet. To record one, run make decide

REPORT=poc-main/reports/daily/2026-11-10/urllib3-5370 DECISION=approve (or reject,

approve_with_notes, re_run).

Rerun command

make check PR=https://github.com/urllib3/urllib3/pull/5370

Evidence links

• Pull request urllib3/urllib3#5370

• record.json

• pr.diff

• setup-main.log

• setup-pr.log

• git.log
• pytest-main-1.log
• pytest-pr-1.log
• pytest-main-2.log
• pytest-pr-2.log
• pytest-main-3.log
• pytest-pr-3.log
• junit-main-1.xml
• junit-pr-1.xml
• coverage-pytest.log
• coverage-pr.json
• probe-model-probe-c1-read-main-1.jsonl
• probe-model-probe-c1-read-pr-1.jsonl
• probe-model-probe-c1-read1-main-1.jsonl
• probe-model-probe-c1-read1-pr-1.jsonl
• probe-model-probe-c3-main-1.jsonl
• probe-model-probe-c3-pr-1.jsonl
• probe-model-probe-c5-main-1.jsonl
• probe-model-probe-c5-pr-1.jsonl
• main run 1 result: pytest-main-1.log:80 «196 passed, 47 skipped in 3.52s»
• pr run 1 result: pytest-pr-1.log:80 «200 passed, 47 skipped in 3.60s»

Glossary

| Term | Meaning

| main | The library as it is today, without the PR.

| PR | Pull request: a proposed change to the library.

| run | One full pass of the selected tests. We do 3 per side, alternating main and PR.

| flaky | A test that gives different results when nothing changed; it is ignored as evidence.

| log line | A line in a pytest-*.log file: the raw output the verdict is based on.

| route | How the two sides were built: nightly+patch, source-build or source-pair (see docs/OPERATIONS.md).

| changed lines covered | How many of the PR's changed, executable Python lines ran during the PR side's tests.

| re-parametrized | A test whose parameter ids changed (renamed cases); the new ids are judged on their own.

rebuilt from cleaned text

Written 2026-11-10 17:39 IST